ANNOTATED CODE DOCUMENT

CIVE 202 – Project #4: Interpret (FEMA NRI Sensitivity Analysis)

Mina Awad, Ava Johnson, Elizabeth Wiggins

University of Nebraska–Omaha

April 21, 2026

Section 1: Importing Libraries

Python Code	Annotations
`import pandas as pd`	This line imports the pandas library and assigns it the alias pd. Pandas is used for loading, cleaning, merging, and manipulating tabular data throughout the analysis.
`import numpy as np`	This line imports the NumPy library with the alias np. NumPy provides support for NaN handling and numerical operations used in the alternative risk score calculation.
`import matplotlib.pyplot as plt`	This line imports the pyplot module from matplotlib with the alias plt. Matplotlib is the primary plotting library used to generate all bar charts, heatmaps, and scatter plots in this analysis.
`import matplotlib.patches as mpatches`	This line imports the patches module from matplotlib. Patches are used to create custom legend entries for the categorical choropleth maps, avoiding the PatchCollection legend warning.
`import seaborn as sns`	This line imports the seaborn library with the alias sns. Seaborn is built on top of matplotlib and is used specifically for the cross-tabulation heatmaps in this analysis.
`import geopandas as gpd`	This line imports the GeoPandas library with the alias gpd. GeoPandas extends pandas to support geospatial data and is used to read shapefiles and generate choropleth maps.

Section 2: Loading the NRI and SVI Datasets

Python Code	Annotations
`nri_ca = pd.read_csv( 'NRI_Table_CensusTracts_California.csv', low_memory=False)`	This line uses pd.read_csv() to load the NRI Census Tract dataset for California into a DataFrame named nri_ca. The file contains 9,106 rows and 467 columns. The low_memory=False parameter prevents mixed data type warnings on large files with many columns.
`nri_co = pd.read_csv( 'NRI_Table_CensusTracts_Colorado.csv', low_memory=False)`	This line loads the NRI Census Tract dataset for Colorado into a DataFrame named nri_co. The file contains 1,447 rows and 467 columns covering all Census Tracts across 64 Colorado counties.
`svi_ca = pd.read_csv('California.csv', low_memory=False)`	This line loads the ATSDR Social Vulnerability Index dataset for California into a DataFrame named svi_ca. The file contains 9,109 rows and 158 columns, including the overall SVI percentile RPL_THEMES.
`svi_co = pd.read_csv('Colorado.csv', low_memory=False)`	This line loads the ATSDR SVI dataset for Colorado into a DataFrame named svi_co. The file contains 1,447 rows and 158 columns.
`county_ca = pd.read_csv( 'NRI_Table_Counties_California.csv', low_memory=False)`	This line loads the NRI county-level dataset for California (58 counties, 465 columns) into a DataFrame. County-level data is used later to compare risk ratings across geographic scales.
`county_co = pd.read_csv( 'NRI_Table_Counties_Colorado.csv', low_memory=False)`	This line loads the NRI county-level dataset for Colorado (64 counties, 465 columns) for the same scale comparison analysis.

Section 3: Data Exploration

Python Code	Annotations
`key_cols = ['TRACTFIPS', 'STATE', 'COUNTY', 'RISK_SCORE', 'RISK_RATNG', 'EAL_SPCTL', ...]`	This line defines a list of the key NRI column names that will be inspected during data exploration. These columns represent the core risk variables needed for the sensitivity analysis.
`print(nri_ca[key_cols].head())`	This line uses the .head() function to display the first 5 rows of selected key columns from the California NRI dataset, allowing visual verification that the data loaded correctly.
`print(nri_ca['RISK_RATNG'] .value_counts())`	This line uses the .value_counts() function to count and display the frequency of each NRI risk rating category for California Census Tracts. The result shows California is skewed toward Relatively Moderate (3,418) and Relatively High (2,639).
`print(svi_ca[svi_cols].head())`	This line displays the first 5 rows of key SVI columns for California, including RPL_THEMES and the four theme percentiles, to verify data structure before merging.
`print((svi_ca['RPL_THEMES'] == -999).sum())`	This line counts the number of California tracts where RPL_THEMES equals -999, the SVI sentinel value for missing data. The result is 68 tracts that will need to be excluded from the alternative risk calculation.

Section 4: Data Cleaning and Merging NRI + SVI

Python Code	Annotations
`svi_ca_merge = svi_ca[['FIPS', 'RPL_THEMES', 'RPL_THEME1', 'RPL_THEME2', 'RPL_THEME3', 'RPL_THEME4']].copy()`	This line selects only the six columns needed from the SVI dataset for merging and creates an independent copy using .copy(). This avoids the SettingWithCopyWarning and keeps the merge operation clean.
`merged_ca = nri_ca.merge( svi_ca_merge, left_on='TRACTFIPS', right_on='FIPS', how='left', suffixes=('', '_SVI'))`	This line performs a left join between the NRI and SVI DataFrames using the 11-digit Census Tract FIPS code as the join key. The left join retains all 9,106 NRI tracts and adds SVI columns where a FIPS match exists. The suffixes parameter prevents duplicate column name conflicts.
`merged_ca.loc[ merged_ca[col] == -999, col ] = np.nan`	This line replaces -999 sentinel values with NumPy NaN inside a for loop iterating over all SVI theme columns. Using .loc[] ensures the replacement modifies the DataFrame in place without triggering a chained assignment warning.
`ca_valid = merged_ca.dropna( subset=['RPL_THEMES', 'EAL_SPCTL']).copy()`	This line uses .dropna() with a subset argument to remove tracts with missing RPL_THEMES or EAL_SPCTL values, then creates a clean copy. California drops from 9,106 to 9,041 valid tracts (65 excluded); Colorado drops from 1,447 to 1,431 (16 excluded).

Section 5: Alternative Risk Score Calculation

Python Code	Annotations
`ca_valid['ALT_RISK_SCORE'] = 0.6 * ca_valid['EAL_SPCTL'] + 0.4 * (ca_valid['RPL_THEMES'] * 100)`	This line computes the alternative risk score using the weighted additive formula. EAL_SPCTL is already on a 0–100 scale; RPL_THEMES is on a 0–1 scale and is multiplied by 100 to match. The 0.6/0.4 weights reflect the client's emphasis on hazard exposure as the primary risk driver.
`ca_valid['ALT_RISK_PCTL'] = ca_valid['ALT_RISK_SCORE'] .rank(pct=True) * 100`	This line uses the pandas .rank(pct=True) function to convert each tract's alternative risk score into a within-state percentile ranking from 0 to 100. This ensures the alternative ratings are relative to other tracts in the same state, matching the NRI's approach.
`def classify_risk(pctl): if pctl <= 20: return 'Very Low' elif pctl <= 40: return 'Relatively Low' elif pctl <= 60: return 'Relatively Moderate' elif pctl <= 80: return 'Relatively High' else: return 'Very High'`	This function classifies a percentile value into one of five qualitative risk categories using the same breakpoints that FEMA applies to the NRI: Very Low (≤20th), Relatively Low (20th–40th), Relatively Moderate (40th–60th), Relatively High (60th–80th), and Very High (>80th percentile).
`ca_valid['ALT_RISK_RATNG'] = ca_valid['ALT_RISK_PCTL'] .apply(classify_risk)`	This line uses the .apply() function to call classify_risk on every tract's alternative percentile value, creating a new column ALT_RISK_RATNG with the qualitative risk category. This column is directly compared against the NRI's RISK_RATNG.

Section 6: Cross-Tabulation Tables

Python Code	Annotations
`rating_order = ['Very Low', 'Relatively Low', 'Relatively Moderate', 'Relatively High', 'Very High']`	This line defines a list specifying the correct ordinal sequence of risk rating categories from lowest to highest. This list is used throughout the analysis to ensure consistent ordering in tables, figures, and comparisons.
`ca_xtab = pd.crosstab( ca_valid['RISK_RATNG'], ca_valid['ALT_RISK_RATNG'], margins=True, margins_name='Total')`	This line uses pd.crosstab() to create a contingency table comparing NRI ratings (rows) against alternative ratings (columns) for all 9,041 valid California tracts. The margins=True parameter adds row and column totals. Diagonal values represent tracts where both methods agree.
`ca_xtab = ca_xtab.reindex( index=rating_order + ['Total'], columns=rating_order + ['Total'], fill_value=0)`	This line reorders the cross-tabulation rows and columns to follow the logical rating sequence from Very Low to Very High, with the Total row/column last. The fill_value=0 fills any categories that have no tracts.
`ca_agree = (ca_valid['RISK_RATNG'] == ca_valid['ALT_RISK_RATNG']).sum()`	This line counts the number of tracts where the NRI and alternative ratings are exactly the same by comparing the two columns element-wise. California agreement: 4,336 of 9,041 tracts (48.0%). Colorado agreement: 445 of 1,431 tracts (31.1%).

Section 7: Figure 1 — Rating Distribution Bar Chart

Python Code	Annotations
`fig, axes = plt.subplots(1, 2, figsize=(14, 6))`	This line creates a figure with two side-by-side subplots (1 row, 2 columns) at 14 inches wide and 6 inches tall. The left subplot displays California and the right subplot displays Colorado.
`nri_counts = data['RISK_RATNG'] .value_counts() .reindex(rating_order, fill_value=0)`	This line counts the frequency of each NRI rating category and reorders them using the predefined rating_order list. The fill_value=0 ensures categories with no tracts still appear in the chart.
`ax.bar(x - width/2, nri_counts.values, width, label='NRI Rating', color='#2166ac', alpha=0.8)`	This line draws the NRI rating bars offset to the left of each x-position. The blue color (#2166ac) distinguishes NRI bars from the alternative rating bars. Alpha=0.8 adds slight transparency.
`ax.bar(x + width/2, alt_counts.values, width, label='Alternative Rating', color='#b2182b', alpha=0.8)`	This line draws the alternative rating bars offset to the right, creating the grouped bar chart effect. The red color (#b2182b) contrasts with the blue NRI bars for clear visual comparison.
`plt.savefig( 'fig1_rating_distribution.png', dpi=150, bbox_inches='tight')`	This line saves the completed figure as a PNG file at 150 DPI resolution. The bbox_inches='tight' parameter removes excess whitespace around the figure for cleaner output.

Section 8: Figure 2 — Cross-Tabulation Heatmap

Python Code	Annotations
`inner = xtab.loc[ rating_order, rating_order]`	This line selects only the five risk rating rows and columns from the cross-tabulation table, excluding the Total row/column, to create a clean 5×5 matrix for the heatmap.
`sns.heatmap(inner, annot=True, fmt='d', cmap='YlOrRd', ax=ax, xticklabels=short_labels, yticklabels=short_labels)`	This line uses sns.heatmap() to create an annotated heatmap of the cross-tabulation matrix. annot=True displays the cell values, fmt='d' formats them as integers, and cmap='YlOrRd' applies a yellow-orange-red colormap where darker colors indicate higher tract counts.

Section 9: Figure 3 — EAL vs. SVI Scatter Plot

Python Code	Annotations
`ax.scatter( subset['EAL_SPCTL'], subset['RPL_THEMES'] * 100, c=colors[rating], label=rating, alpha=0.3, s=5)`	This line plots each Census Tract as a point with EAL percentile on the x-axis and SVI percentile (scaled to 0–100) on the y-axis. Points are colored by NRI risk rating using the five-color diverging scheme. Alpha=0.3 adds transparency to reveal overlapping density, and s=5 keeps point size small for the large California dataset.

Section 10: Categorical Bias Analysis

Python Code	Annotations
`def rating_to_num(r, order=rating_order): return order.index(r) if r in order else -1`	This function converts a qualitative risk rating string into a numeric index (0=Very Low through 4=Very High) so that the direction of disagreement between NRI and alternative ratings can be calculated arithmetically.
`upgraded = (data['ALT_NUM'] > data['NRI_NUM']).sum()`	This line counts the number of tracts where the alternative method assigns a higher (more severe) risk category than the NRI. California: 1,624 upgraded. Colorado: 976 upgraded (68.2% of all tracts).
`downgraded = (data['ALT_NUM'] < data['NRI_NUM']).sum()`	This line counts tracts where the alternative method assigns a lower risk category. California: 3,081 downgraded. Colorado: only 10 downgraded (0.7%). This asymmetry in Colorado is the strongest evidence of categorical bias.

Section 11: GeoPandas Mapping

Python Code	Annotations
`tracts = gpd.read_file( 'Student Shape Files')`	This line uses gpd.read_file() to load the NRI national Census Tract shapefile into a GeoDataFrame. The file contains polygon geometries and all 467 NRI data columns for every U.S. Census Tract. Loading takes 2–5 minutes due to the 748 MB file size.
`tracts.crs = 'EPSG:3857'`	This line manually assigns the correct coordinate reference system (Web Mercator) to the GeoDataFrame. The NRI shapefile coordinates are in meters, which corresponds to EPSG:3857.
`tracts = tracts.to_crs('ESRI:102003')`	This line reprojects all geometries from Web Mercator to Albers Equal Area Conic (ESRI:102003), the standard projection for mapping the contiguous United States that preserves area relationships.
`state_map.plot( column='RISK_SCORE', ax=ax, legend=True, linewidth=0.0, cmap='RdYlBu_r', ...)`	This line uses the GeoDataFrame .plot() method to create a choropleth map where each Census Tract polygon is colored by its NRI Risk Score. The RdYlBu_r colormap assigns red to high-risk tracts and blue to low-risk tracts. linewidth=0.0 removes polygon borders for cleaner visualization.
`handles = [mpatches.Patch( color=rating_colors[r], label=r) for r in rating_order]`	This line creates custom legend patches for categorical choropleth maps using a list comprehension. Each patch is a colored rectangle matching the risk rating color scheme. This approach avoids the matplotlib PatchCollection legend warning that occurs with the default .plot() legend.

Section 12: Export Cleaned Data

Python Code	Annotations
`ca_valid.to_csv( 'California_NRI_SVI_Merged _Clean.csv', index=False)`	This line uses the .to_csv() function to export the cleaned and merged California dataset (9,041 tracts with alternative risk scores) to a CSV file. The index=False parameter excludes the DataFrame index from the output for cleaner data.
`co_valid.to_csv( 'Colorado_NRI_SVI_Merged _Clean.csv', index=False)`	This line exports the cleaned Colorado dataset (1,431 tracts) in the same format. Both exported files serve as reproducible intermediate outputs that can be independently verified.